

StockMate — Progress Report

A simple stock tracker for a small motel • C#, Windows Forms, SQLite

Aashir Bhusal | s2400901

ITS203 Assessment C — Milestone 2 (5%)

Due Week 10 — 20 September 2026

1. Work completed so far

StockMate is the motel stock tracker I proposed in Milestone 1. The class structure from the proposal is built, the program runs, and five of my six main requirements work from start to finish with data saved in a SQLite file. The code is split into three folders so each part has one job: `Models` holds the classes and the rules, `Data` holds all the SQL, and `Forms` holds the windows. No form contains SQL, and no form can change a quantity directly.

Component	Kind	What it does now
StockItem	Abstract class	Fields shared by every item, validated in the setters (a blank name or a negative cost throws an <code>ArgumentException</code>). <code>QuantityOnHand</code> has a private setter and only changes through <code>Receive()</code> and <code>Issue()</code> . Declares abstract <code>NeedsAttention()</code> and <code>AttentionMessage()</code> .
ConsumableItem	Subclass	Adds a use-by date. Needs attention when it is low, within 30 days of expiry, or expired, and reports the most urgent problem first.
DurableItem	Subclass	Adds a replacement interval in months and a last-replaced date. Needs attention when it is low or past its replacement date.
IReportable	Interface	One method, <code>ToReportLine()</code> , implemented by <code>StockItem</code> and overridden by both subclasses.
InsufficientStockException	Custom exception	Thrown by <code>Issue()</code> when more is requested than is on hand. Carries the item name, the quantity requested and the quantity available.
Database	Data class	Creates the <code>StockItems</code> and <code>Transactions</code> tables, builds the correct subclass from each row using the <code>ItemType</code> column, uses named parameters, and saves a movement and the new quantity in one database transaction.
MainForm	Form	Stock grid, category filter and the "Needs attention" list, which is filled on start-up by calling <code>NeedsAttention()</code> on a <code>List<StockItem></code> without checking the type.
ItemForm	Form	Adds or edits an item. Shows the use-by field or the replacement fields depending on the type. When editing, the type and quantity are locked.
MovementForm	Form	Records a receipt or an issue, with the quantity and the staff member's name.

Table 1 — Classes and forms built so far

ID	Requirement from my proposal	Status	Where it stands
FR-01	Add a consumable or durable item	Working	<code>ItemForm</code> builds the right subclass and <code>InsertItem()</code> saves it.
FR-02	Edit an item, or turn it off	Partly	Editing works. <code>IsActive</code> is in the model and the database and the list hides inactive items, but no button sets it yet.
FR-03	Record a delivery (receipt)	Working	Adds to the quantity and saves the date and staff name.
FR-04	Record an issue; refuse over-issue	Working	See Figure 2. Issuing exactly what is on hand is allowed and ends at zero.
FR-05	Item table with category filter	Working	See Figure 1. The filter fills itself from the categories in use.
FR-06	Warning list on start-up	Working	Low stock, expiring within 30 days, expired, and due for replacement.
FR-07	History for one item (extra)	Not started	Movements are saved to <code>Transactions</code> , but nothing reads them back yet.
FR-08	Export stock list to CSV (extra)	Not started	<code>TotalValue</code> is already worked out on each item, ready for the export.

Table 2 — Progress against the functional requirements in my Milestone 1 proposal

Figure 1 — Main window with test data and all four kinds of warning

Figure 2 — Refusing to issue more than is on hand

In Figure 2 I tried to take out 100 bath towels when only 62 were on hand. `Issue()` threw `InsufficientStockException`, and `MainForm` caught it in its own catch block, placed before the general catch (`Exception`), so the user gets a plain message. The quantity and the database were not changed.

2. Challenges encountered

Setting up .NET without admin rights. The computer I develop on only had the .NET 6 runtime, so nothing would build, and I do not have administrator rights to run the normal installer. I found that Microsoft's install script can put the .NET 8 SDK into my own user folder instead, which fixed the build. The problem came back in a second form later: the built StockMate.exe refused to start when opened directly and asked me to install .NET, because Windows does not look in a per-user folder by itself. Running it with `dotnet run`, or setting the `DOTNET_ROOT` variable first, fixes it. *Next step:* I will test on the machine I will present from well before Week 13, not on the day.

My database design was more complicated than the problem. The proposal had three tables, including a separate Categories lookup table. When I wrote the Database class I saw that this meant a join on every read and a whole extra form just to manage categories, for no real benefit at this size. I dropped it and made category a text column on StockItems; the filter box now fills itself with a `SELECT DISTINCT` query. I learned that a design that looks tidy on paper still has to earn its place once there is code to write for it.

Working in large blocks instead of steadily. My commit history shows that I have built this project in a few long sessions rather than a little each week. It worked, but it is risky: a problem found late leaves no time to fix it, and large commits are harder to describe and to undo. *Next step:* from Week 11 each feature in the plan below gets its own commit on the day I finish it, so the history follows the work.

3. Plan for remaining weeks

When	What I will do	Finished when
Week 11 21–27 Sep	Finish FR-02: a "Deactivate item" button with an "are you sure?" prompt, which sets <code>IsActive</code> to false so the item leaves the list but keeps its history. Add <code>GetHistory(itemId)</code> to Database and a history window for one item (FR-07). Fix a layout bug found while preparing this report: on the movement form the question label overlaps the quantity box and hides the first digits.	A retired item disappears from the list and its movements can still be viewed.
Week 12 28 Sep – 3 Oct	Add the CSV export with a total stock value (FR-08). Run my test scenarios: issue more than on hand, issue exactly what is on hand, blank name, expiry dates either side of the 30-day limit, a durable past its replacement date, and close and reopen the program. Fix what they find, finish the README and write the Milestone 3 reflection.	Every scenario has a recorded result and the repository is final.
Week 13 5–11 Oct	Rehearse the demonstration and the OOP questions on the presentation machine. No new features.	The demo runs from start to finish without notes.

Table 3 — Plan to the final submission

Risk	What I will do about it
The history window takes longer than one week	Week 11 holds only this and the deactivate button. If it overruns into Week 12, FR-08 is the feature I drop.
Testing finds problems too late to fix	I will test each feature in the week I build it, and keep Week 12's scenario run for the whole program.
The program will not start on the presentation machine	Check for the .NET 8 runtime on that machine in Week 12, and keep the exact start-up steps in the README.

Table 4 — Risks to the plan

Changes from the proposal. Nothing has been removed. FR-07 and FR-08 were marked as extras and I still expect to deliver both; if time runs short I will drop FR-08 first, as the proposal said. Two things have changed. The Categories table was dropped, as described above. Also, the proposal said everything would sit in one window with tabs, but I built one main window with small pop-up forms instead, because adding an item and recording a movement are short tasks and a pop-up keeps the stock list in view behind them.

4. GitHub repository link

<https://github.com/AashirBhusal/StockMate>

The repository is public. It holds the complete Visual Studio solution, a README with run instructions (`dotnet run --project StockMate`, or open `StockMate.sln` and press F5) and my references and tools, my development notes, my Milestone 1 proposal and this report, and a `.gitignore` that keeps build output, the local database and secrets out of Git.